

JAVA CONSOLE APPLICATION

Blood Bank Inventory Donor Matcher

A Java console application verifying donor–recipient blood group compatibility and recording clinical checks.

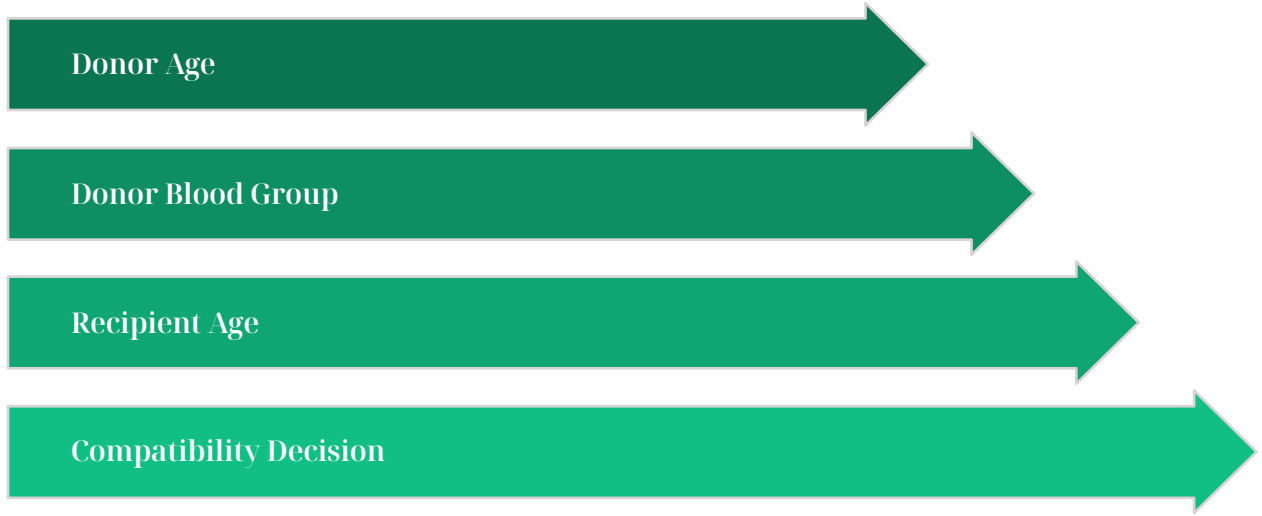
By: Mohd Imaad (2620040060)
K Sri Akshay (2620090093)

Made with GAMMA

ABSTRACT

Blood Bank Inventory Donor Matcher

Blood Bank Inventory Donor Matcher is a menu-driven Java console application that checks whether a donor's blood can be given to a recipient. It validates donor age (18 to 65) and recipient age (0 to 120), applies ABO and Rh compatibility rules, and reports CAN or CANNOT donate. Every check is kept in memory and saved to a text file, so the history is still there after the program is closed. Wrong inputs, such as letters instead of numbers or a lowercase blood group, are handled without crashing the program.



PRE-CHECK AGE VALIDATION

Donor eligibility: ages 18–65 · Recipient limits: ages 0–120 · Applied prior to cross-matching.

Made with GAMMA

CURRENT-STATE REVIEW

Existing Manual System & Operational Vulnerabilities

MANUAL REFERENCE

Manual Paper Charts

Compatibility verification relies on printed reference charts and staff memory.

HUMAN FACTOR

Human Error Risk

Manual evaluation is slow, fatigue-prone, and lacks automated fail-safes during emergencies.

ELIGIBILITY

No Eligibility Gates

Donor age and health restrictions are not systematically enforced before testing.

RECORDS

Fragmented Record Keeping

Physical paper registers make audit trails and historical lookups slow and unreliable.

Solution: an automated Java program that validates inputs, checks compatibility and keeps a permanent record of every check.

Made with GAMMA

Proposed Java Architecture & System Flow

Core program flow

- **Menu-driven console:** A do-while menu repeats until Exit.
- **Age validation:** While loops re-ask donor age 18–65 and recipient age 0–120 until valid, and invalid input such as letters is caught with try-catch and asked again.
- **Donor group logic:** An if-else-if ladder identifies the donor's blood group from 8 types.
- **Recipient logic:** A switch statement sets the compatible result.

Results and storage

- **Stored results:** An ArrayList holds the records, each check is appended to records.txt, and old records load at startup.
- **Input safety:** Blood groups are accepted in any case (trim and toUpperCase).
- **Recursion option:** The user can check another pair, and the recursive call continues only when the user answers Y.
- **Overloaded printRecord():** CANNOT results can include a detailed diagnostic header.

Made with GAMMA

CO 1 · MODULE 1

Algorithm, Structure & Basic I/O

01

Algorithm

Input, validate, decide, store, repeat.

02

Structure

One class with main() as the entry point.

03

Data Types

int menuChoice and ages, boolean compatible, String names and result.

04

Constants

static final FILE_NAME and VALID_GROUPS.

05

Operators

Relational <, >; logical ||, !; assignment =.

06

Console I/O

Scanner nextLine(), System.out.print and print

Made with GAMMA

CO 2 · MODULE 2

Control Flow & Decisions

01

if-else-if ladder

Identifies the donor's blood group from 8 types and prints whom it can donate to.

02

switch

Main menu (1, 2, 3) and recipient blood group decision, with break and default.

03

while loops

Re-ask donor age (18 to 65), recipient age (0 to 120) and blood group until valid.

04

do-while loop

The menu appears at least once and repeats until the user picks Exit.

05

Compound conditions

The age check uses age < 18 || age > 65.

Made with GAMMA

CO 3 · MODULE 3

Methods, Arrays & Recursion

01

Methods & parameters

`checkCompatibility(sc, recordList)`, `readInt(sc, prompt)`

02

Return values

`readInt` returns `int`, `readBloodGroup` returns `String`, `isValidGroup` returns `boolean`.

03

Method overloading

`printRecord(result)` and `printRecord(result, true)` adds a detailed header for CANNOT results.

04

Recursion

`checkCompatibility` calls itself when the user answers Y; base case N.

05

1D array & search

`VALID_GROUPS` array searched one by one in `isValidGroup()`.

06

Enhanced for-each

Prints all past records and scans `VALID_GROUPS`.

Made with GAMMA

CO 4 · MODULE 4

Exceptions, Static Members & Objects

- **Exception handling (try-catch):** `readInt()` catches `NumberFormatException`, so typing letters no longer crashes the program; it just asks again.
- **Checked vs unchecked:** `NumberFormatException` is unchecked (runtime); `IOException` from the file methods is checked, so Java forces us to handle it with try-catch.
- **Static members and constants:** `static final FILE_NAME` and `VALID_GROUPS`; all methods are static so `main()` can call them directly.
- **Objects and the new keyword:** `Scanner` and `ArrayList<String>` objects are created with `new` and passed around as references.

```
public static int readInt(Scanner sc, String prompt) {
    while (true) {
        System.out.print(prompt);
        try {
            return Integer.parseInt(sc.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.println("Invalid input. Please enter a whole number.");
        }
    }
}
```

Made with GAMMA

CO 5 · MODULE 5

Strings and File I/O

01

String methods

`trim` removes spaces; `toUpperCase` makes a+ and A+ the same; `equals` and `equalsIgnoreCase` compare text; `+` joins text.

02

equals vs ==

`equals` compares the actual text; `==` compares whether both variables point to the same object, so we use `equals`.

03

Immutability

Strings never change; `trim` and `toUpperCase` return a new String, so we assign the result back.

04

File I/O

`newBufferedWriter` with `CREATE` and `APPEND` adds each check to records.txt; `readAllLines` loads old records at startup.

05

Safe file handling

`Files.exists` handles a missing file, `IOException` is caught, and try-with-resources closes the file.

Made with GAMMA

CO 6 · MODULE 6

Collections & Streams

01

List collection (ArrayList)

`ArrayList<String> recordList` stores every check, growing automatically, so there is no fixed limit like the old array of 100.

02

Generics (typed collection)

`<String>` means only Strings can be added; mistakes are caught when compiling, and no casting is needed when reading records back.

03

Operations used

`add()` stores a new check, `isEmpty()` detects "no checks yet", `addAll()` loads old records into the list.

04

Iteration & sharing

The enhanced for-each loop prints all past checks; `recordList` is passed to `checkCompatibility`, so every check is kept in one list.

COVERAGE

Used: List collection, Generics, add/isEmpty/addAll, for-each

Partly used: reading files into a List (`Files.readAllLines`)

Not used: HashMap, Autoboxing, Lambdas, Streams

```
ArrayList<String> recordList = new ArrayList<>();
loadRecordsFromFile(recordList);   // fills the list from records.txt
recordList.add(result);          // store a new check
for (String record : recordList) { printRecord(record); }
```

Made with GAMMA

CONCLUSION

What We Built and Learned

01

Result

A working menu-driven program that validates donor and recipient details, checks ABO/Rh compatibility and keeps a permanent history in records.txt.

02

Learning

Control flow, methods and recursion, exception handling, String methods and file handling in one project.

03

Limitations

Console only; recursion is used for repeat checks where a loop would be safer; records are plain text.

04

Future scope

Donor class objects, sorted records, blood stock quantities, and a GUI or database version.

Made with GAMMA

DEMONSTRATION PATH

System Execution & Console Workflow

Three simple figures show the program from menu selection to compatibility results and past checks.

FIGURE 1

Main menu

Blood Bank Inventory Donor System

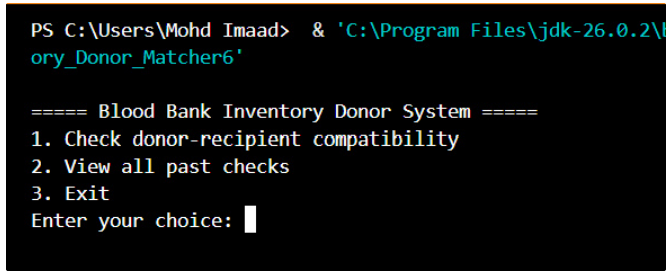


FIGURE 2

Compatibility result

Donor–recipient

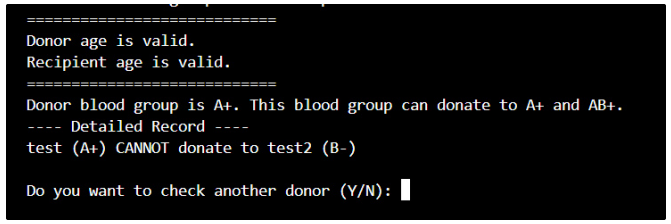
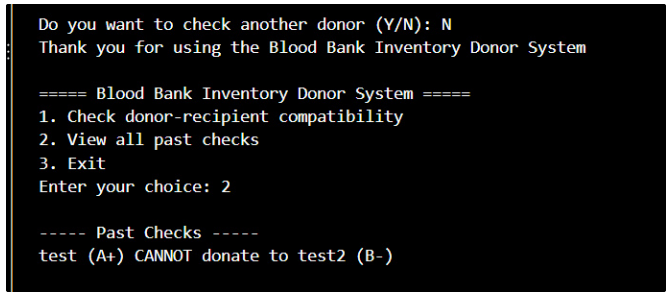


FIGURE 3

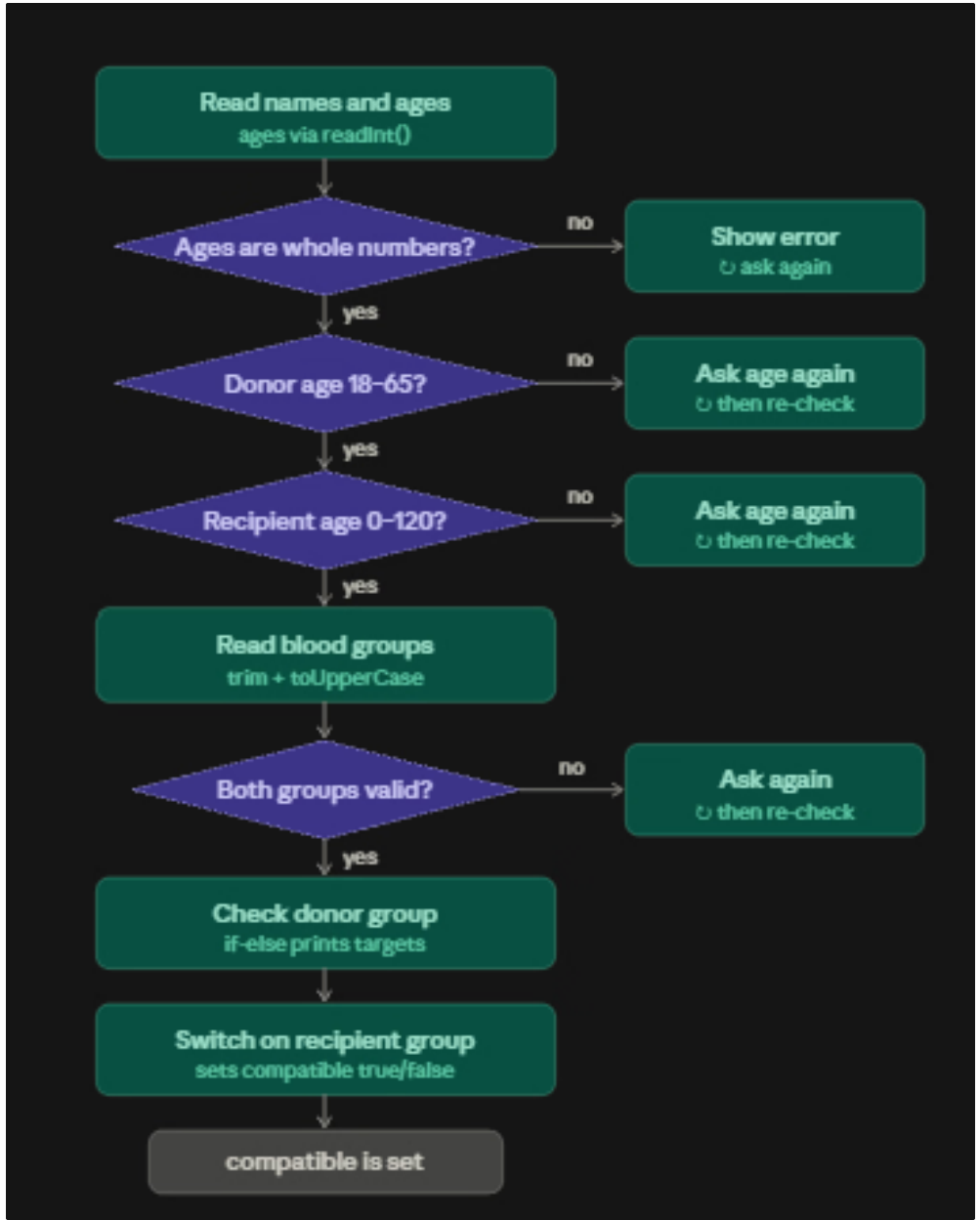
Viewing past checks

Record history



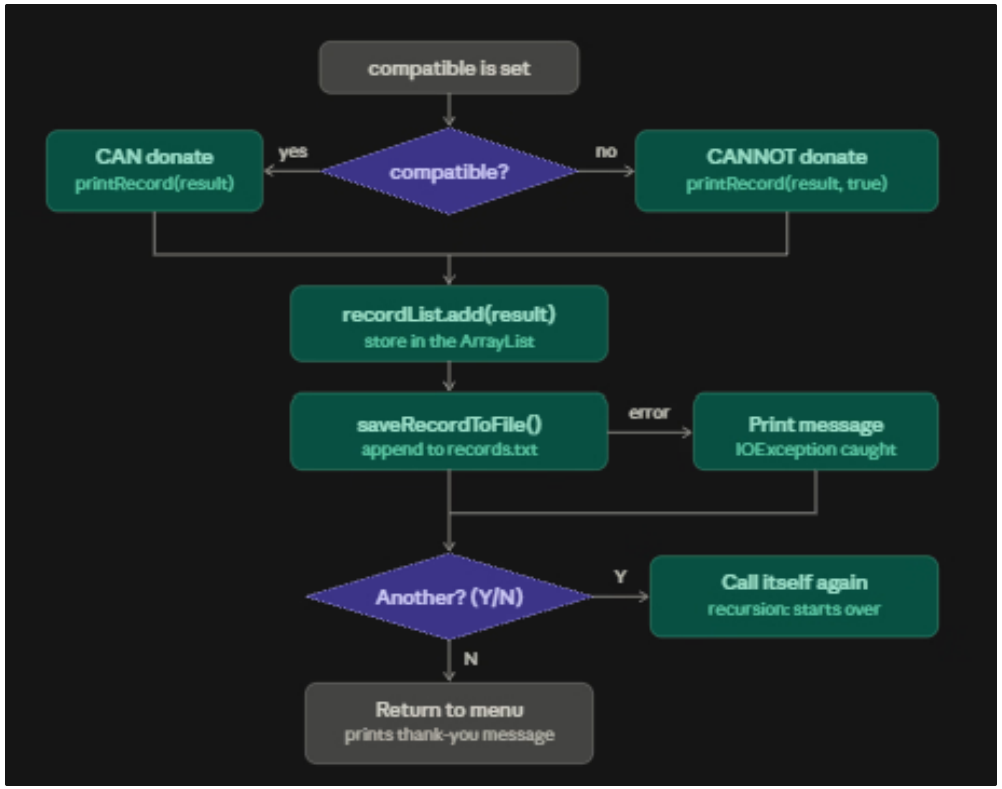
Made with GAMMA

Flowchart for the Program



Part 2: Compatibility Evaluation, Record Storage and Recursive Calling

- **Storage:** each result is added to the ArrayList and appended to records.txt; IOException prints a message.
- **Recursion loop:** continues only when the user answers Y.



Made with GAMMA